Can Traditional Programming Bridge the Ninja Performance Gap for Parallel Computing Applications?

Nadathur Satish†, Changkyu Kim†, Jatin Chhugani†, Hideki Saito⋆, Rakesh Krishnaiyer⋆, Mikhail Smelyanskiy†, Milind Girkar⋆, and Pradeep Dubey†

†Parallel Computing Lab, Intel Corporation ⋆Intel Compiler Lab, Intel Corporation

ABSTRACT

Current processor trends of integrating more cores with wider SIMD units, along with a deeper and complex memory hierarchy, have made it increasingly more challenging to extract performance from applications. It is believed by some that traditional approaches to programming do not apply to these modern processors and hence radical new languages must be discovered. In this paper, we question this thinking and offer evidence in support of traditional programming methods and the performance-vs-programming effort effectiveness of common multi-core processors and upcoming many-core architectures in delivering significant speedup, and close-to-optimal performance for commonly used parallel computing workloads.

We first quantify the extent of the *"Ninja gap"*, which is the performance gap between naively written C/C++ code that is parallelism unaware (often serial) and best-optimized code on modern multi-/many-core processors. Using a set of representative throughput computing benchmarks, we show that **there is an average *Ninja gap* of 24X (up to 53X)** for a recent 6-core Intel® Core™ i7 X980 Westmere CPU, and that this gap if left unaddressed will inevitably increase. We show how a set of well-known algorithmic changes coupled with advancements in modern compiler technology can bring down the Ninja gap to an average of **just 1.3X**. These changes typically require low programming effort, as compared to the very high effort in producing Ninja code. We also discuss hardware support for programmability that can reduce the impact of these changes and even further increase programmer productivity. We show equally encouraging results for the upcoming Intel® Many Integrated Core architecture (Intel® MIC) which has more cores and wider SIMD. We thus demonstrate that we can contain the otherwise uncontrolled growth of the Ninja gap and offer **a more stable and predictable performance growth** over future architectures, offering strong evidence that radical language changes are not required.

1. INTRODUCTION

Performance scaling across processor generations has previously relied on increasing clock frequency. Programmers could ride this trend and did not have to make significant code changes for improved code performance. However, clock frequency scaling has hit the power wall [32], and the free lunch for programmers is over.

Recent techniques for increasing processor performance have a focus on integrating more cores with wider SIMD units, while simultaneously making the memory hierarchy deeper and more complex. While the peak compute and memory bandwidth on recent processors has been increasing, it has become more challenging to extract performance out of these platforms. This has led to the situation where only a small number of expert programmers ("Ninja programmers") are capable of harnessing the full power of modern multi-/many-core processors, while the average programmer only obtains a small fraction of this performance. We define the term "Ninja gap" as the performance gap between naively written parallelism unaware (often serial) code and best-optimized code on modern multi-/many-core processors.

There have been many recent publications [45, 43, 14, 2, 28, 33] that show 10-100X performance improvements for real-world applications through adopting highly optimized platform-specific parallel implementations, proving that a large Ninja gap exists. This typically requires high programming effort and may have to be re-optimized for each processor generation. However, these papers do not comment on the effort involved in these optimizations. In this paper, we aim at quantifying the extent of the Ninja gap, analyzing the causes of the gap and investigating how much of the gap can be bridged with low effort using traditional C/C++ programming languages.[1]

We first quantify the extent of the Ninja gap. We use a set of real-world applications that require high throughput (and inherently have a large amount of parallelism to exploit). We choose throughput applications because they form an increasingly important class of applications [13] and because they offer the most opportunity for exploiting architectural resources - leading to large Ninja gaps if naive code does not take advantage of these resources. We measure performance of our benchmarks on a variety of platforms across different generations: 2-core Conroe, 4-core Nehalem, 6-core Westmere, Intel® Many Integrated Core (Intel® MIC), and the NVIDIA C2050 GPU. Figure 1 shows the Ninja gap for our benchmarks on three CPU platforms: a 2.4 GHz 2-core E6600 Conroe, a 3.33 GHz 4-core Core i7 975 Nehalem and a 3.33 GHz 6-core Core i7 X980 Westmere. The figure shows that there is up to a 53X gap between naive C/C++ code and best-optimized code for a recent 6-core Westmere CPU. The figure also shows that this gap has been increasing across processor generations - the gap is 5-20X on a 2-core Conroe system (average of 7X) to 20-53X on Westmere (average of 25X). This gap has been increasing in spite of micro-architectural improvements that have reduced the need and impact of performing various optimizations.

We next analyze the sources of the large performance gap. There are a number of reasons why naive code performs badly. First, the code may not be parallelized, and compilers do not automatically identify parallel regions. This means that the increasing core count is not utilized in the naive code, while the optimized code takes full advantage of it. Second, the code may not be vectorized, leading to under-utilization of the increasing SIMD widths. While auto-

[1]Since measures of ease of programming such as programming time or lines of code are largely subjective, we show code snippets with the code changes required to achieve performance.